Ben Gurion University of the Negev

Principals of Programming Languages
202.1.2051

# ASSIGNMENT 2 - QUESTIONS 1 AND 2D

*Authors:*
Tamar Shefi Simchon (212550966)
Stav Mordekhai (213250400)

May 22, 2026

# Question 1.1

**Claim:** All programs in $L_1$ can be transformed into an equivalent program in $L_{11}$.

**Proof/Explanation:** Let $P_1$ be a program in $L_1$, and let $V_1$ be the set of all variables defined using a `define` expression in $P_1$.

In the evaluation semantics of $L_1$, when a `define` expression is encountered, the value of its body (the third argument) is fully evaluated before the identifier binding occurs. Because this evaluation resolves to a static value prior to binding, we can construct an equivalent program $P_{11}$ in $L_{11}$ by applying the following structural transformations:

1. **Variable Substitution:** Every free occurrence of a variable $v \in V_1$ located outside of its original `define` expression is replaced directly with the evaluated body text of $v$'s definition.
2. **Definition Removal:** All explicit `define` statements for the variables in $V_1$ are then safely removed from the program.

Since every occurrence of a variable $v$ is structurally substituted with an expression that evaluates to the exact same value as its original binding, the semantic behavior and final computed output of $P_{11}$ is strictly equivalent to $P_1$. Therefore, any program in $L_1$ can be translated into an equivalent program in $L_{11}$.

# Question 1.2

**Claim:** There exists a program $P_2$ in $L_2$ which cannot be transformed into an equivalent program in $L_{21}$.

**Proof/Counterexample:** Consider the following recursive program in $L_2$:

```
(define x (lambda () (x)))
(x)
```

**Explanation:**

- **In $L_2$:** The `define` expression evaluates to a closure. When `(x)` is applied, the identifier `x` is successfully resolved via environment lookup, enabling a valid structural recursion.
- **In $L_{21}$:** Because variable definitions are eliminated via direct substitution, transforming this program requires unfolding `x` infinitely within the lambda body:

  ```
  ((lambda () ((lambda () (...)) )))
  ```

Without a global environment containing the defined variables, an anonymous lambda cannot reference itself. Thus, $P_2$ cannot be translated into a finite, equivalent program in $L_{21}$.

# Question 1.3

**Claim:** All programs in $L_2$ can be transformed into an equivalent program in $L_{22}$.

**Proof/Construction:** Any multi-parameter lambda abstraction in $L_2$ can be transformed into an equivalent sequence of single-parameter lambdas in $L_{22}$. We define this transformation inductively using the following algorithm, where $P$ is the list of parameters and $B$ is the expression body:

**reduce_lambda**$(P, B)$:

1. **if** $|P| \leq 1$: **return** `(lambda (`$P$`)` $B$`)`
2. **else**: Let $x_1, x_2, \ldots, x_n$ be the parameters in $P$.
   **return** `(lambda (`$x_1$`) reduce_lambda((`$x_2, \ldots, x_n$`),` $B$`))`

By structural induction, this transformation preserves the evaluation semantics of the original program, ensuring that any program in $L_2$ can be structurally mapped to an equivalent program in $L_{22}$.

## Question 1.4

**Claim:** There exists a program $P_2$ in $L_2$ which cannot be transformed into an equivalent program in $L_{23}$.

**Proof/Counterexample:** Consider the following function composition program in $L_2$:

```
(lambda (f g)
  (lambda (x)
    (f (g x))))
```

**Explanation:** The evaluation of this program results in a higher-order closure that encapsulates function composition ($f \circ g$).

- **In $L_2$:** Functions are first-class. This expression evaluates to a valid closure that accepts two arbitrary functions, $f$ and $g$, as parameters and returns a new functional closure `(lambda (x) ...)` as its runtime value.
- **In $L_{23}$:** Procedures are strictly first-order, meaning they are completely forbidden from accepting functions as arguments or returning functions as values.

Since $L_{23}$ cannot bind parameters to closures or evaluate a procedure that yields a new closure, this program has no structural equivalent in $L_{23}$.

## Question 1.5

Special forms can't be implemented simply as primitive operators due to their distinct evaluation rules.

Consider the case of the `define` expression, which is mandatory for the expressive completeness of $L_2$. If we assume for contradiction that `define` is a standard primitive operator rather than a special form, its behavior would break in two fundamental ways under standard evaluation semantics:

1. **Applicative-Order Evaluation:** Primitive operators evaluate all of their arguments *before* the operator itself is applied. For an expression like `(define x 5)`, a primitive operator would attempt to evaluate the symbol `x` before binding it, resulting in an error.
2. **Environment Mutation:** By definition, a pure primitive operator takes input values, executes a computation, and returns a final value without introducing side-effects to the surrounding context. It inherently lacks the semantic capability of a `define` expression to mutate and expand the global environment frame.

## Question 1.6

**Claim:** A function body with multiple expressions is not required in pure functional programming. Because $L_3$ forbids internal variable definitions and side-effects, supporting multiple expressions is unnecessary.

**Explanation:** In pure functional programming, functions are free of side-effects. If a function body contains a sequence of expressions $e_1, e_2, \ldots, e_n$:

- The evaluation of preceding expressions ($e_1$ through $e_{n-1}$) cannot mutate state or alter the environment.
- Since the final return value depends solely on $e_n$, preceding expressions act as isolated code. The only semantic impact they can have is if they trigger a runtime error or cause the program to enter an infinite loop.

Therefore, if we wish to intentionally crash a program or force an infinite loop for specific inputs in $L_3$, multiple expressions are necessary. Otherwise, a single expression body is entirely sufficient.

Multi-expression bodies are primarily useful in imperative, procedural, and object-oriented paradigms. In those environments, preceding expressions are executed specifically for their side-effects—such as mutating memory, altering state, or performing I/O—before returning the final value.

# Question 1.7

**Definition:** A lexical address is a static attribute of a variable reference in a program that determines the specific declaration scope to which that reference is bound.
**Explanation:** The lexical address is formally represented as a tuple `[var : depth pos]`, where:

- **`var`:** The identifier name of the variable.
- **`depth`:** The number of lexical contours (environment frames) that must be crossed upward to reach the variable's declaration context.
- **`pos`:** The structural offset or index of the variable within that specific frame.

When an identifier name is duplicated across contours, the reference statically resolves to the inner-most binding layer within the Abstract Syntax Tree. Language primitives are not declared within the program and are thus treated as *free* variables.

### Example Analysis

Consider the following program:

```
1: (define y 2)
2: (lambda (x) (+ x y))
```

Inside the body of the `lambda` expression on line 2, the occurrences of `x` and `y` resolve as follows:

1. **The Variable `x`:** The identifier `x` is bound locally as a parameter of the immediate `lambda` block. Because it resides in the current frame, its lexical distance is 0.
2. **The Variable `y`:** The identifier `y` is free relative to the local `lambda` expression. Going up towards the global environment contour where it was declared on line 1, resulting in a lexical distance of 1.

Consequently, the lexical addresses for the program is resolved as follows:

```
1: (define [y : 0 0] [2 free])
2: (lambda (x) ([+ free] [x : 0 0] [y : 1 0]))
```

# Question 1.8

**Trade-offs in Primitive Operator Representation:**

- **Representation as `primOp`:** Optimizes evaluation performance. Primitives map directly to meta-language procedures, and do not require global environment lookup.
- **Representation as Closures:** Maximizes interpreter extensibility. Adding a new primitive requires no structural changes to the interpreter. It simply requires adding its binding to the initial global environment.

# Question 1.9

**a.** Switching from normal-order evaluation to applicative-order evaluation is justified when a parameter appears multiple times within the body of a `lambda` expression and is passed as a complex expression at application. For example:

```
(define f (lambda (x) (+ x x x x x x x x x x)))
(f (* (+ 1 3) (- 8 (+ 2 4))))
```

Evaluating this in normal-order would lead to evaluating the complex expression `(* (+ 1 3) (- 8 (+ 2 4)))` 10 distinct times—once for each occurrence of `x` in the body of `f`. In applicative-order, there will be a single evaluation of this argument expression prior to function application, substituting the parameter with a pre-computed primitive number.

**b.** Switching from applicative-order evaluation to normal-order evaluation is justified when the operands are already simple. In such cases, evaluating the arguments offers no operational benefit. For example:

```
(define f (lambda (x y z w)
            (if x
                (+ y z w)
                (* y z w))))
(f #t 2 3 4)
```

In applicative-order, the simple expressions `#t`, `2`, `3`, and `4` are evaluated into primitive boolean and numeric values, which are then immediately re-wrapped into their corresponding literal expressions. In normal-order, because we substitute the raw expressions of the operands directly into the body without prior evaluation, this literal conversion overhead is entirely avoided.

# Question 1.10

**a.** In $L_3$, the role of the `valueToLit` function is to convert the runtime values resulting from an `AppExp` evaluation back into literal expressions, enabling them to be substituted directly into the body of the function. This conversion is strictly necessary because the function body is structurally a `CEXP`, which by syntactic definition can only contain core expressions and cannot contain raw, naked values.

**b.** Since normal-order evaluation does not evaluate operands prior to substitution, the arguments are already structurally in the form of a `CEXP`. Consequently, they can be directly substituted into the body of the function without requiring any intermediate `valueToLit` transformation.

**c.** In the environment-model interpreter, the `valueToLit` function is completely unnecessary because the evaluator does not rely on text-based code substitution to execute function application. Instead, application is performed by extending the lexical scope—creating a new environment frame where the parameters are cleanly bound directly to their evaluated values. Any variable references inside the function body are then resolved dynamically via a standard environment lookup chain, completely bypassing the need to map values back into literal AST nodes.

# Question 1.11

**a.** Renaming is not required in the environment model since variable references are evaluated as a lookup in the bindings of the current environment in a hierarchical order. This means that its value would be the first appearance of its binding, from the lowest frame up.

**b.** Renaming is **not** required in the substitution model if the term being substituted is a "closed" term.
The purpose of renaming bound variables during substitution is to prevent **free variable capture**. Variable capture occurs when a *free* variable in the incoming operand expression falls into the scope of a bound variable with the same name inside the function body. Variable capture is impossible when substituting a closed term. Since a closed term contains no free variables by definition, it has no unbound identifiers that could accidentally be captured by internal binders within the function body.

## Question 2d

**Convert the ClassExps to ProcExps:**

```
(define pi 3.14)

(define square (lambda (x) (* x x)))

(define circle
  (lambda (x y radius)
    (lambda (msg)
      (if (eq? msg 'area)
          (* (square radius) pi)
          (if (eq? msg 'perimeter)
              (* 2 pi radius)
              'error)))))

(define c (circle 0 0 3))

(c 'area)
```

**List the expressions which are passed to the `L3ApplicativeEval` function during the computation of the program (after the conversion) for the case of substitution model:**

```
3.14
(lambda (x) (* x x))
(lambda (x y radius) ...)
(circle 0 0 3)
circle
0
0
3
(lambda (msg) ...)
(c 'area)
c
'area
(if (eq? 'area 'area) ...)
(eq? 'area 'area)
eq?
'area
'area
(* (square 3) pi)
*
(square 3)
square
3
(* 3 3)
*
3
3
pi
```

**Draw the environment diagram for the computation of the program (after the conversion) for the case of the environment model interpreter**